
Health Wearable Device

Firmware Architecture & Code Analysis Report

Band01_Rev1 · Project Work — Task 2

Prepared by	Masrur Jamil Prochchod
Institution	Hochschule Hamm-Lippstadt (HSHL)
Date of revision	20 May 2026
Repository	github.com/giovannix/smartband_v001_firmware_masrur
Hardware reference	PCB Component Analysis Report — Band01_Rev1 (Task 1)

Table of Contents

1. Introduction	4
2. Development Environment	4
2.1 Required tools and SDKs.....	4
3. Repository Structure	4
4. Boot Sequence — Startup/main.c	5
4.1 Key initialisation steps	5
4.2 Code snippet — main().....	5
4.3 Startup/ccfg_app_ble.c	6
5. Application Layer — Application/	6
5.1 simple_peripheral.c — Main Application Task (1,647 lines)	6
Key constants defined at the top of simple_peripheral.c.....	6
SimplePeripheral_init() — Task initialisation	6
SmartBand_periodicFxn() — Core sensor and BLE update loop.....	7
Button handling — buttonCallbackFxn()	8
5.2 i2c_sensors.c — Sensor I2C Driver (437 lines).....	8
I2C transaction pattern.....	8
Sensor functions	9
Temperature conversion — MAX30205	9
5.3 oled.c — SSD1306 Display Driver (334 lines).....	9
oledInit() — Initialisation sequence.....	9
Public API	10
Display layout at startup	10
5.4 oled_thread.c — Dedicated OLED Task (183 lines).....	10
5.5 app_thread.c — Secondary Application Task	10
5.6 Supporting files.....	11
6. BLE GATT Profile — PROFILES/	11
6.1 simple_gatt_profile.c — GATT Service Definition	11
How data flows from sensor to phone	11
6.2 Other PROFILES/ files.....	12
6.3 BLE security and pairing.....	12
7. BLE Stack and Memory Map.....	12
7.1 Stack library.....	12
7.2 Memory map.....	13
8. RTOS Architecture — SYS/BIOS Task Model.....	13

9. Power Architecture in Firmware	14
10. Debugging and Assertions	15
11. Firmware–Hardware Correspondence.....	15
12. Current Status and Roadmap.....	16
12.1 Steps to activate live sensor reading	16
12.2 Future work driven by the research paper	17
13. Conclusion	17
References.....	18

1. Introduction

This report fulfils the requirements of Task 2 of the Project Work at Hochschule Hamm-Lippstadt (HSHL). Having completed the PCB Component Analysis Report (Task 1), which documented all 52 hardware components of the Band01_Rev1 smartband, this report now examines the firmware that runs on that hardware.

The firmware is built for the Texas Instruments CC2640R2F microcontroller using the TI SimpleLink CC2640R2 SDK and TI BLE Stack, developed in Code Composer Studio (CCS). The high-resolution heart-rate estimation algorithm proposed in that paper (ESPRIT via Hilbert Transform) is the longer-term signal-processing target for the firmware; the current build provides the BLE, sensor-driver and OLED infrastructure on which that algorithm will later run.

2. Development Environment

The firmware was developed and is intended to be built using Texas Instruments Code Composer Studio (CCS), version 10 or newer. CCS is an Eclipse-based integrated development environment specifically designed for TI microcontrollers. The required tools are summarised below.

2.1 Required tools and SDKs

Component	Version / Details
Texas Instruments Code Composer Studio (CCS)	Version 10 or newer (Eclipse-based IDE)
TI SimpleLink CC2640R2 SDK	Matching version used in project (BLE stack + drivers)
TI ARM Compiler Toolchain	Included with CCS installation
TI BLE Stack (FlashROM)	Pre-compiled, stored in FlashROM_StackLibrary/
SYS/BIOS RTOS	Real-time operating system, included in SDK
XDS110 Debug Probe	Hardware JTAG/SWD probe for flashing and debugging
Git / GitHub	Version control; repository cloned via GitHub invitation

Although source files may be viewed in any editor such as Visual Studio Code, compilation, linking, flashing, and SDK path configuration must be performed through CCS. The project configuration files .project, .cproject, and .ccsproject must not be removed as they define all CCS build settings, include paths, and compiler flags.

3. Repository Structure

The repository contains 70 files organised into the following directories. Each folder serves a distinct architectural purpose within the firmware.

Folder / File	Purpose
Startup/	Boot sequence, chip configuration, and RTOS entry point
Application/	Main application logic, sensor drivers, OLED driver, BLE peripheral task

Folder / File	Purpose
PROFILES/	BLE GATT service and characteristic definitions
Include/	Shared header files including BLE bond manager (gapbondmgr.h)
FlashROM_StackLibrary/	Pre-compiled TI BLE protocol stack (not modified by user)
TOOLS/	SYS/BIOS configuration script (app_ble.cfg) and RTOS ROM library
targetConfigs/	CCS debug target configuration for CC2640R2F
BOM/	Aisler Bill of Materials PDF (March 2026)
.project / .cproject / .ccsproject	CCS project configuration files (required, do not delete)
.gitignore	Excludes build artefacts (.hex, .out, .map) from version control

4. Boot Sequence — Startup/main.c

The file Startup/main.c is the firmware entry point. It is the first code executed when the CC2640R2F is powered on or reset. Its function is to initialise low-level hardware, set up the inter-task communication layer, create all RTOS tasks with their assigned priorities, and hand execution to the SYS/BIOS scheduler permanently.

4.1 Key initialisation steps

The eight calls executed in main() before BIOS_start() are, in order:

- RegisterAssertCback(AssertHandler) — registers the error handler for BLE stack assertions.
- PIN_init(BoardGpioInitTable) — initialises all GPIO pins (I2C, buttons, JTAG, RF switch).
- VIMSMODESet(VIMS_BASE, VIMS_MODE_ENABLED) — enables instruction cache prefetching for performance.
- ICall_init() — initialises the ICall inter-task messaging layer between application and BLE stack.
- ICall_createRemoteTasks() — starts the BLE radio stack tasks (highest internal priority).
- GAPRole_createTask() — creates the GAP (Generic Access Profile) BLE role task at Priority 5.
- SimplePeripheral_createTask() — creates the main application task at Priority 2.
- BIOS_start() — starts the SYS/BIOS RTOS scheduler; this call never returns.

4.2 Code snippet — main()

```
int main()
{
    RegisterAssertCback(AssertHandler);
    PIN_init(BoardGpioInitTable);
    VIMSMODESet(VIMS_BASE, VIMS_MODE_ENABLED);

    ICall_init();
    ICall_createRemoteTasks();           // BLE stack – Priority 5 (highest)
    GAPRole_createTask();               // GAP role – Priority 5
    SimplePeripheral_createTask();       // App task – Priority 2
    //App_createTask();                 // DISABLED in current build
    //OledThread_createTask();          // DISABLED in current build

    BIOS_start();                       // Starts RTOS – never returns
    return 0;
}
```

```
}
```

4.3 Startup/ccfg_app_ble.c

This file configures the Customer Configuration (CCFG) area of the CC2640R2F flash memory, which stores chip-level settings including BLE stack configuration, bootloader behaviour, and power settings. It is compiled and flashed alongside the application but is separate from the main program flow.

5. Application Layer — Application/

The Application/ folder contains the main firmware logic. It is the layer that sits between the hardware drivers and the BLE stack, implementing the device's actual behaviour: reading sensors, updating the display, handling button presses, and transmitting data wirelessly.

5.1 simple_peripheral.c — Main Application Task (1,647 lines)

This is the largest and most important file in the firmware (53 KB, 1,647 lines). It implements the SimplePeripheral RTOS task that runs at Priority 2 and is the core of all device behaviour. It is based on TI's SimplePeripheral BLE example application, extended by the project team to add sensor reading, OLED display, and button handling.

Key constants defined at the top of simple_peripheral.c

Constant	Value	Meaning
DEFAULT_ADVERTISING_INTERVAL	160	BLE advertising interval ($160 \times 0.625 \text{ ms} = 100 \text{ ms}$)
DEFAULT_DESIRED_MIN_CONN_INTERVAL	80	Minimum connection interval ($80 \times 1.25 \text{ ms} = 100 \text{ ms}$)
DEFAULT_DESIRED_MAX_CONN_INTERVAL	800	Maximum connection interval ($800 \times 1.25 \text{ ms} = 1000 \text{ ms}$)
DEFAULT_DESIRED_CONN_TIMEOUT	1000	Connection supervision timeout ($1000 \times 10 \text{ ms} = 10 \text{ s}$)
SBP_PERIODIC_EVT_PERIOD	350	Sensor polling and BLE update interval (350 ms)
SBP_TASK_PRIORITY	2	RTOS task priority (lower than BLE stack at 5)
SBP_TASK_STACK_SIZE	800	Stack size in bytes allocated for this task

SimplePeripheral_init() — Task initialisation

Called once when the task first runs, this function configures all BLE parameters, registers callbacks, initialises GATT services, sets up the button GPIO interrupt handler, and initialises the OLED display.

```
// From SimplePeripheral_init() in simple_peripheral.c (line 620)
// Set up periodic clock – fires SmartBand_periodicFxn every 350 ms
Util_constructClock(&periodicClock, SimplePeripheral_clockHandler,
    SBP_PERIODIC_EVT_PERIOD, 0, false, SBP_PERIODIC_EVT);

// Configure button GPIO pins with pull-up and falling-edge interrupt
Board_BUTTON0 | PIN_INPUT_EN | PIN_PULLUP | PIN_IRQ_NEGEDGE,
Board_BUTTON1 | PIN_INPUT_EN | PIN_PULLUP | PIN_IRQ_NEGEDGE,
```

```
// Initialise OLED display
oledInit();
oledFill(0x00); // Clear screen
oledWriteString(0, 0, "Smart Band V001"); // Splash screen
```

SmartBand_periodicFxn() — Core sensor and BLE update loop

This function is the heartbeat of the firmware. It is triggered every 350 ms by a SYS/BIOS Clock object. It checks the BLE connection state, reads sensor data (or placeholder data in the current build), packages the data into a 6-byte array, and updates the BLE GATT characteristics so connected devices receive the latest values.

```
// From SmartBand_periodicFxn() in simple_peripheral.c (line 496)
void SmartBand_periodicFxn(void)
{
    uint16_t i2c_signal_data;
    gaprole_States_t state;
    uint8_t state8;

    GAPRole_GetParameter(GAPROLE_STATE, &state);
    state8 = (button1State << 4 | button0State); // Pack button states

    if ((state == GAPROLE_CONNECTED) | (state == GAPROLE_CONNECTED_ADV))
    {
        // --- CURRENT STATE (placeholder) ---
        i2c_signal_data = 0x7788; // Hardcoded test value

        // --- TO BE ACTIVATED ---
        // i2c_signal_data = get_i2c_value_max30205_rawtemp(0x48); // Temp

        uint8_t upper = i2c_signal_data >> 8;
        uint8_t lower = i2c_signal_data;

        // --- FULL SENSOR ARRAY (commented out, ready to activate) ---
        // uint8_t array[] = {get_i2c_value_max3010x_id(),
        //                     get_i2c_value_lis331_whoami(),
        //                     get_i2c_value_lis331_xh(),
        //                     get_i2c_value_lis331_xl(),
        //                     upper, lower};

        uint8_t array_simple_ble[] = {0, 1, 2, 0, upper, lower};

        // Push 6 bytes to BLE CHAR1 (UUID 0xFFF1)
        SimpleProfile_SetParameter(SIMPLEPROFILE_CHAR1,
                                   SIMPLEPROFILE_CHAR1_LEN,
                                   array_simple_ble);

        // Push button state to BLE CHAR2 (UUID 0xFFF2)
        SimpleProfile_SetParameter(SIMPLEPROFILE_CHAR2, 1, &state8);
    }

    // Re-enable button interrupts (debounce)
    PIN_setConfig(buttonPinHandle, PIN_BM_IRQ,
                  Board_BUTTON0 | PIN_IRQ_NEGEDGE);
    PIN_setConfig(buttonPinHandle, PIN_BM_IRQ,
                  Board_BUTTON1 | PIN_IRQ_NEGEDGE);
}
```

Current status: the actual sensor calls (MAX30102, LIS331, MAX30205) are commented out and replaced with placeholder data {0, 1, 2, 0, 0x77, 0x88}. The infrastructure — I2C drivers, BLE

characteristics, periodic clock — is fully in place. Activating live sensor reading requires uncommenting and testing the sensor function calls; the minimal set of edits is summarised in §12 (Status and Roadmap) and is not repeated here.

Button handling — `buttonCallbackFxn()`

Button presses are handled via GPIO interrupts, registered in `SimplePeripheral_init()`. When SW101 (DIO_0) or SW102 (DIO_1) is pressed, the interrupt fires and the callback toggles the button state variable. The state (both buttons packed into one byte) is transmitted via CHAR2 every 350 ms.

```
// buttonCallbackFxn – called from hardware interrupt context
static void buttonCallbackFxn(PIN_Handle handle, PIN_Id pinId)
{
    PIN_setConfig(handle, PIN_BM_IRQ, pinId | PIN_IRQ_DIS); // Disable (debounce)
    switch (pinId)
    {
        case Board_BUTTON0:
            button0State = (button0State == 0) ? 1 : 0;    // Toggle
            break;
        case Board_BUTTON1:
            button1State = (button1State == 0) ? 1 : 0;    // Toggle
            break;
    }
}
// Note: interrupt is re-enabled at the end of SmartBand_periodicFxn()
```

5.2 `i2c_sensors.c` — Sensor I2C Driver (437 lines)

This file contains all low-level I2C communication functions for the three sensors. Every function follows the same pattern: initialise an `I2C_Transaction` struct with the target register address, open the I2C bus at 400 kHz, perform the transfer, close the bus, and return the result. The sensors correspond exactly to the hardware components documented in the PCB report (Task 1).

I2C transaction pattern

```
// Standard I2C read pattern (from i2c_sensors.c)
I2C_Handle      handle;
I2C_Params      params;
I2C_Transaction i2c_trans;

uint8_t tx_buf[1] = { register_address };
uint8_t rx_buf[1];

I2C_Params_init(&params);
params.bitRate = I2C_400kHz;    // Fast mode (400 kHz)

i2c_trans.writeCount  = 1;    // Send: register address
i2c_trans.writeBuf    = tx_buf;
i2c_trans.readCount   = 1;    // Receive: 1 byte
i2c_trans.readBuf     = rx_buf;
i2c_trans.slaveAddress = SENSOR_ADDR;    // e.g. 0x57 for MAX30102

handle = I2C_open(Board_I2C0, &params); // DIO_5 = SCL, DIO_6 = SDA
if (I2C_transfer(handle, &i2c_trans))
    return rx_buf[0];

I2C_close(handle);
```


Sensor functions

Function	Sensor	I2C addr	Action
get_i2c_value_max3010x_id()	MAX30102 (UIC1)	0x57	Reads PART_ID register (0xFF) — identifies chip
get_i2c_value_max3010x_basicinit()	MAX30102 (UIC1)	0x57	Sets LED1 to 6.2 mA; sets mode to red-only (0x02)
get_i2c_value_lis331_powerup()	LIS331 (U103)	0x18	Writes CTRL_REG1 = 0b11000111 (normal power, all axes)
get_i2c_value_lis331_whoami()	LIS331 (U103)	0x18	Reads WHO_AM_I register (0x0F) — returns 0x33 ¹
get_i2c_value_lis331_xh()	LIS331 (U103)	0x18	Reads X-axis high byte (register 0x29)
get_i2c_value_lis331_xl()	LIS331 (U103)	0x18	Reads X-axis low byte (register 0x28)
get_i2c_value_max30205_rawtemp(addr)	MAX30205 (U104)	0x48–0x4F	Reads 2-byte temperature register 0x00; returns raw 16-bit value

¹ The WHO_AM_I value 0x33 is taken from the firmware constant. The STMicroelectronics datasheet for LIS331DL gives 0x3B, and LIS331DLH/HH give 0x32 — neither matches 0x33. This discrepancy should be verified against the actual STM datasheet for the exact LIS331 variant fitted on Band01_Rev1 before flashing; if the constant is wrong, the WHO_AM_I sanity check at boot will silently fail.

Temperature conversion — MAX30205

The MAX30205 returns a 16-bit two's-complement value in two bytes. The firmware reads both bytes and combines them. The raw value is converted to degrees Celsius using the formula specified in the datasheet: Temperature (°C) = raw_value × 0.00390625. This gives the 0.00390625 °C (1/256 °C) resolution and ±0.1 °C accuracy noted in the PCB report.

```
// From get_i2c_value_max30205_rawtemp() – returns raw 16-bit value
uint16_t value = (rx_buf[0] << 8 | rx_buf[1]); // MSB first

// Conversion to Celsius (application-layer responsibility):
float temp_celsius = raw_value * 0.00390625f;
```

5.3 oled.c — SSD1306 Display Driver (334 lines)

This file implements a lightweight driver for the SSD1306 128×64 OLED display connected via I2C (address 0x3C). The driver was originally written by Ricardo K. Miranda in 2019. It provides six public functions used throughout the application.

oledInit() — Initialisation sequence

```
// From oledInit() in oled.c – SSD1306 init sequence
uint8_t buff1[] = {0xae, 0xd5, 0x00, 0xa8}; // Display off, clock, mux
oledWriteCommandn(buff1, 4);
oledWriteCommand(HEIGHT - 1); // 63 (rows 0..63 = 64 rows)

uint8_t buff2[] = {0xd3, 0x00, 0x40, 0x8d}; // Offset, start line, charge pump
oledWriteCommandn(buff2, 4);
oledWriteCommand(0x14); // Charge pump ON

uint8_t buff5[] = {0xdb, 0x40, 0xa4, 0xa6, 0x2e, 0xaf};
oledWriteCommandn(buff5, 6); // VCOMH, normal, deactivate scroll,
```

```
// display ON (0xaf)
```

Public API

Function	Description
oledInit()	Sends the 22-byte SSD1306 initialisation sequence (close to the Adafruit reference).
oledFill(uint8_t data)	Fills all 8 pages × 128 columns with the given byte. Called with 0x00 to clear the screen.
oledWriteString(int x, int y, char *str)	Looks up each character in ucFont[] (8 bytes per character) and writes 8 pixel columns per character.
oledDrawPic(pic[], w, h, x, y)	Draws a binary bitmap image stored in flash. Images must be stored rotated 90° clockwise.
oledSetPosition(int x, int y)	Sends three I2C commands to position the write cursor: page select, lower column, upper column.
oledShutdown()	Sends command 0xAE to turn off the OLED panel.

Display layout at startup

When the firmware boots successfully, the OLED displays the splash screen: the text "Smart Band V001" is written at position (0,0) using `oledWriteString()`. In the commented-out full implementation, the medical-cross icon and live sensor values would appear on subsequent lines.

5.4 oled_thread.c — Dedicated OLED Task (183 lines)

This file implements a dedicated SYS/BIOS task for OLED display management using an event-driven queue architecture. Although currently disabled in `main.c` (commented out), the implementation is complete and demonstrates the intended threading model for display updates.

```
// oled_thread.c – Task priorities and timing constants
#define OLED_TASK_STACK_SIZE    800    // Stack in bytes
#define OLED_TASK_PRIORITY      1      // Lowest priority (display is non-critical)
#define OLED_FPS_PERIOD         ...    // Screen refresh interval (from header)
```

The task waits on `OLED_EVENT_INIT` before initialising the display, then enters a loop waiting for `OLED_EVENT_SCREEN_UPDATE` (triggered by a periodic clock). Messages are posted to `oledQueue` from other tasks using `Util_enqueueMsg()`, and the display task dequeues and processes them. Supported message types are: `MEDIC_PIC` (draw medical-cross icon), `HEART_PIC` (draw heart icon), `BUTTON_PRESS` (show button state), and `DATA_HEX` (show new sensor data).

5.5 app_thread.c — Secondary Application Task

A second application task is defined in `app_thread.c`, also currently disabled in `main.c`. It is intended to run at Priority 1 alongside the OLED thread and was designed to handle sensor reading and queue messages to the OLED thread. In the current build, its sensor call (`get_i2c_value_bmp085_temp`) is set to return 0, indicating the architecture is a scaffold awaiting live sensor integration. Once enabled, the intended split of responsibilities is: `simple_peripheral` handles BLE, `app_thread` handles sensor sampling and signal processing, `oled_thread` handles display updates. The three tasks communicate via SYS/BIOS Events and Queues.

5.6 Supporting files

Several smaller files in Application/ provide shared utilities and resources.

File	Purpose
fonts.c	Contains the ucFont[] character bitmap array used by oledWriteString(). Each ASCII character occupies 8 bytes (5×8 font padded to 8 wide).
images.c	Contains binary bitmap data for the heart and medical-cross icons (heart1_big[], medic[], etc.).
util.c / util.h	TI utility functions: Util_constructClock(), Util_startClock(), Util_enqueueMsg(), Util_dequeueMsg().
att_rsp.c	Handles ATT (Attribute Protocol) response buffering for BLE — part of TI BLE stack glue code.
rcosc_calibration.c	Optional: calibrates the internal RC oscillator as an alternative to the 32 kHz crystal Y2 for sleep timing.

6. BLE GATT Profile — PROFILES/

The PROFILES/ folder defines the Bluetooth Low Energy data structure that is exposed to connected devices such as smartphones. This is implemented using the GATT (Generic Attribute Profile) framework, which organises data into services and characteristics.

6.1 simple_gatt_profile.c — GATT Service Definition

The device exposes one primary GATT service with UUID 0xFFFF0, containing five characteristics. This service is what a smartphone BLE scanner sees when connecting to the band.

Characteristic	UUID	Length	Access	Current use
CHAR1	0xFFFF1	6 bytes	Read + Write	Sensor data array: [MAX3010x_id, LIS_whoami, LIS_xh, LIS_xl, temp_upper, temp_lower]
CHAR2	0xFFFF2	1 byte	Read only	Status byte: bit[0]=button0State, bit[4]=button1State, plus BLE connection state
CHAR3	0xFFFF3	1 byte	Read + Write	General purpose — reserved for future sensor data
CHAR4	0xFFFF4	1 byte	Notify	Push-notification channel — sends alerts to phone without polling
CHAR5	0xFFFF5	5 bytes	Read + Write	Extended data buffer — reserved for future multi-byte sensor values

How data flows from sensor to phone

```
// 1. Sensor read (i2c_sensors.c) — every 350 ms
uint8_t id      = get_i2c_value_max3010x_id();
uint8_t whoami  = get_i2c_value_lis331_whoami();
uint8_t xh      = get_i2c_value_lis331_xh();
uint8_t xl      = get_i2c_value_lis331_xl();
uint16_t tmp     = get_i2c_value_max30205_rawtemp(0x48);
```

```
// 2. Pack into 6-byte array
uint8_t data[6] = { id, whoami, xh, xl, tmp >> 8, tmp & 0xFF };

// 3. Update BLE characteristic (simple_peripheral.c)
SimpleProfile_SetParameter(SIMPLEPROFILE_CHAR1, 6, data);

// 4. TI BLE stack automatically sends GATT notification
//    to connected phone via 2.4 GHz BLE radio
```

6.2 Other PROFILES/ files

File	Purpose
peripheral.c / peripheral.h	Implements the BLE GAP Peripheral role — manages advertising, connection, and parameter updates.
gatt_uuid.c / gatt_uuid.h	Defines all standard BLE GATT UUIDs (primary service, characteristic, client characteristic configuration).
gatt_profile_uuid.h	Defines custom UUID values for the Simple Profile service (0xFFFF0–0xFFFF5).
gattservapp_util.c	Utility functions for GATT server application management.

6.3 BLE security and pairing

The current build uses the default GAP Bond Manager settings inherited from TI's SimplePeripheral example. Pairing is in Just-Works mode (no MITM protection, no passkey entry), and bonding is disabled by default. Any BLE central that connects can read CHAR1 and CHAR2 without authentication. For a wearable handling clinical data this is acceptable for a research prototype but would need to be revisited before any deployment: either Passkey pairing (display-only on the OLED) or LE Secure Connections with numeric comparison would be appropriate. The relevant configuration is in `gapbondmgr.h` (in `Include/`) and would be applied via `GAPBondMgr_SetParameter()` calls in `SimplePeripheral_init()`.

7. BLE Stack and Memory Map

The CC2640R2F runs the application code and the BLE protocol stack as two separately linked binaries that share the same flash device. Understanding this split is essential for memory budgeting and for explaining why the `FlashROM_StackLibrary/` folder must exist but should not be modified.

7.1 Stack library

The `FlashROM_StackLibrary/` folder contains the pre-compiled Texas Instruments BLE protocol stack stored in a separate region of flash memory. This is not application code and should not be modified. It implements all Bluetooth Low Energy protocol layers below the application:

Protocol layer	Abbreviation	Function
Host Controller Interface	HCI	Command interface between application and BLE radio
Logical Link Control and Adaptation	L2CAP	Multiplexes data channels, handles

Protocol layer	Abbreviation	Function
		segmentation
Security Manager Protocol	SMP	BLE pairing, bonding, and key distribution
Attribute Protocol	ATT	GATT data read/write/notify at byte level
Generic Attribute Profile	GATT	Service and characteristic layer (used by application)
Generic Access Profile	GAP	Advertising, scanning, connection establishment
Radio / RF Driver	—	2.4 GHz BLE radio control via CC2640R2F RF core

7.2 Memory map

The link information files `simple_peripheral_cc2640r2lp_app_linkInfo.xml` and `smart_band_v001_app_linkInfo.xml` define the memory map and linker configuration that separates the application flash region from the BLE stack flash region. These files are generated by CCS and should not be edited manually. The CC2640R2F provides 128 KB of internal flash and 20 KB of SRAM; the typical partitioning produced by the SimpleLink SDK is approximately:

Region	Approx. size	Contents
Application image	~70 KB flash	main, SimplePeripheral task, sensor drivers, OLED driver, GATT profile, util
BLE stack library	~50 KB flash	FlashROM_StackLibrary — HCI, L2CAP, ATT, SMP, GAP, RF driver
CCFG region	88 bytes flash	ccfg_app_ble.c — bootloader, power, debug, BLE radio trim settings
Application SRAM	~12 KB RAM	Heap, task stacks (SimplePeripheral 800 B, OLED 800 B, App 700 B), GATT db
Stack SRAM	~8 KB RAM	Reserved for BLE stack queues, HCI buffers, connection contexts

These figures are SDK defaults; precise numbers for this build can be read from the .map files produced by the linker after a successful build (Debug/smart_band_v001_app.map).

8. RTOS Architecture — SYS/BIOS Task Model

The firmware uses TI's SYS/BIOS real-time operating system, which provides pre-emptive multithreading, hardware interrupt management, inter-task messaging, and timing services. Understanding the task model is essential for working with this codebase.

SYS/BIOS uses fixed-priority pre-emptive scheduling. A higher-priority task always pre-empts a lower-priority task when it becomes ready. Tasks at the same priority are scheduled cooperatively. The BLE stack tasks at Priority 5 will always pre-empt the SimplePeripheral task at Priority 2 when BLE radio events occur — this is by design and ensures BLE timing constraints are always met.

Task	File	Priority	Stack	Status	Responsibility
------	------	----------	-------	--------	----------------

Task	File	Priority	Stack	Status	Responsibility
BLE Stack (remote)	FlashROM_StackLibrary/	5 (highest)	Internal	Active	BLE radio, advertising, connections, protocol stack
GAPRole	PROFILES/peripheral.c	5	Internal	Active	GAP peripheral role, connection events, param updates
SimplePeripheral	Application/simple_peripheral.c	2	800 B	Active	Sensor reads, BLE CHAR updates, OLED init, button handling
OledThread	Application/oled_thread.c	1	800 B	Disabled	Event-driven display updates via queue (commented out)
AppThread	Application/app_thread.c	1	700 B	Disabled	Secondary app task for sensor processing (commented out)

Key RTOS primitives used: Clock (periodic timers), Event (task synchronisation), Queue (inter-task message passing), Semaphore (mutual exclusion), Task (thread definition). All are TI SYS/BIOS constructs, not POSIX.

9. Power Architecture in Firmware

The firmware interacts with the hardware power architecture documented in Task 1. The CC2640R2F manages the LP5907 regulator enable pins via GPIO, allowing sensors to be completely powered down during BLE advertising intervals or deep sleep. This is critical for achieving the ~73-hour battery life on a 150 mAh LiPo battery reported by the research paper.

In the firmware, the POWER_SAVING compile flag controls whether deep sleep (standby) is permitted. When enabled, the CC2640R2F can enter standby between BLE events, consuming below 1 μ A. The 32 kHz crystal Y2 (documented in Task 1) provides the precise low-frequency timing that allows the chip to wake at the correct moment for the next BLE connection event.

Power state	BLE activity	Sensors	Approx. current
Active / Connected	BLE connection active, notifications sent	All powered	~2.0 mA (avg)
Advertising	Periodic BLE advertising (100 ms interval)	Can be gated off	< 1 mA
Deep Sleep	32 kHz crystal Y2 maintains BLE scheduling	All powered down	< 1 μ A

Battery-life sanity check: at an average current draw of 2.032 mA reported by Pinheiro et al. (2021), a 150 mAh LiPo battery yields $150 / 2.032 \approx 73.8$ hours of operation. The 350 ms BLE connection interval used in this firmware is consistent with that figure; shortening the interval would increase radio activity and reduce battery life proportionally.

10. Debugging and Assertions

The firmware exposes two debug paths. The first is the XDS110 JTAG/SWD probe connected via the P1 header (TCK on DIO_3, TMS on DIO_4), which provides full halt/step/inspect capability from CCS. The second is the BLE stack assertion mechanism registered at boot.

RegisterAssertCback(AssertHandler) is the first call in main(). It registers a callback that the BLE stack invokes whenever it detects an internal invariant violation — for example, an HCI command sent in the wrong state, or running out of pre-allocated message buffers. The default AssertHandler in this firmware loops forever, which makes the fault visible to a debugger but invisible to a user with no JTAG attached. A useful improvement (recommended for Task 3) is to extend AssertHandler to write a short error code to the OLED before halting, so faults can be diagnosed without a debugger.

```
// Recommended improvement – visible assertion on OLED
void AssertHandler(uint8 assertCause, uint8 assertSubcause)
{
    char buf[24];
    oledFill(0x00);
    oledWriteString(0, 0, "ASSERT");
    snprintf(buf, sizeof(buf), "cause=%02x sub=%02x", assertCause, assertSubcause);
    oledWriteString(0, 2, buf);
    while (1) { /* spin – JTAG attach to inspect */ }
}
```

11. Firmware–Hardware Correspondence

This section establishes the direct mapping between the firmware code and the physical hardware components documented in the PCB Component Analysis Report (Task 1). Every DIO pin assignment in the firmware corresponds to a physical connection on the Band01_Rev1 PCB.

Hardware component (Task 1)	DIO pin	Firmware reference	Code file
I2C SCL bus (all sensors)	DIO_5	Board_I2C0 (SCL)	i2c_sensors.c, oled.c
I2C SDA bus (all sensors)	DIO_6	Board_I2C0 (SDA)	i2c_sensors.c, oled.c
MAX30102 interrupt (UIC1)	DIO_8	Not yet handled in firmware	simple_peripheral.c
LIS331 interrupt 1 (U103)	DIO_9	Not yet handled in firmware	simple_peripheral.c
LIS331 interrupt 2 (U103)	DIO_2	Not yet handled in firmware	simple_peripheral.c
Push button SW101 (EVQP7J01P)	DIO_0	Board_BUTTON0, buttonCallbackFxn	simple_peripheral.c
Push button SW102 (EVQP7J01P)	DIO_1	Board_BUTTON1, buttonCallbackFxn	simple_peripheral.c
JTAG TCK (P1 header)	DIO_3 / TCK	JTAG debug interface (CCS)	targetConfigs/CC2640 R2F.ccxml
JTAG TMS (P1 header)	DIO_4 / TMS	JTAG debug interface (CCS)	targetConfigs/CC2640 R2F.ccxml
24 MHz crystal Y1 (NX2016SA)	X24M_P / N	Required for BIOS_start(), BLE radio	main.c
32 kHz crystal Y2 (ECS-.327)	X32K_Q1 / Q2	Deep sleep timing, RTC	ccfg_app_ble.c
BLE RF antenna (PCB trace)	RF_P /	BLE stack radio driver	FlashROM_StackLibra

Hardware component (Task 1)	DIO pin	Firmware reference	Code file
	RF_N		ry/

12. Current Status and Roadmap

Based on thorough analysis of the complete source code, the following table summarises the current implementation status of each firmware component.

Component	Status	Notes
BLE advertising and connection	Complete	GAP peripheral role fully operational
GATT service (0xFFFF0) with 5 characteristics	Complete	All UUIDs, permissions, and lengths defined
OLED hardware driver (oled.c)	Complete	SSD1306 init, clear, text, and bitmap drawing all working
Splash screen at boot	Complete	Displays 'Smart Band V001' on OLED at startup
Button GPIO interrupt handling	Complete	Both buttons toggle state; state packed into CHAR2
I2C driver functions for all sensors	Complete	All register-level read/write functions implemented
350 ms periodic clock and BLE update	Complete	Clock fires correctly; SetParameter calls in place
Live MAX30205 temperature reading	Pending	Function exists; call commented out in periodic function
Live MAX30102 heart-rate / SpO2 reading	Pending	Function exists; call commented out in periodic function
Live LIS331 accelerometer reading	Pending	X-axis functions exist; Y/Z and power-up commented out
OLED live sensor display	Pending	oled_thread.c complete but disabled in main.c
Heart-rate ESPRIT algorithm (from paper)	Not yet implemented	Described in research paper; not present in firmware
BLE security (pairing/bonding)	Not yet implemented	Just-Works default; needs hardening for clinical use
OLED thread (oled_thread.c)	Disabled	Complete code exists; needs enabling in main.c
App thread (app_thread.c)	Disabled	Complete code exists; needs enabling in main.c

12.1 Steps to activate live sensor reading

The following minimal changes to simple_peripheral.c are required to activate live sensor data transmission. All driver functions already exist in i2c_sensors.c.

```
// Step 1: Uncomment sensor init in SimplePeripheral_init():
uint16_t lis_status = get_i2c_value_lis331_powerup();
uint16_t max_status = get_i2c_value_max3010x_basicinit();

// Step 2: Replace placeholder in periodicFxn:
```



```
// FROM:
i2c_signal_data = 0x7788;
uint8_t array_simple_ble[] = {0, 1, 2, 0, upper, lower};

// TO:
i2c_signal_data = get_i2c_value_max30205_rawtemp(0x48);
uint8_t array_simple_ble[] = {
    get_i2c_value_max3010x_id(),
    get_i2c_value_lis331_whoami(),
    get_i2c_value_lis331_xh(),
    get_i2c_value_lis331_xl(),
    upper, lower
};

// Step 3: In main.c, uncomment:
OledThread_createTask();
```

12.2 Future work driven by the research paper

The firmware is the embedded implementation of the Multi-Sensor Wearable Health Device (MWHD) framework described by Pinheiro et al. (2021) in *Frontiers in Human Neuroscience* (doi: 10.3389/fnhum.2021.750591). The following items from the paper are the principal extensions that the firmware will require beyond live sensor activation:

- **ESPRIT-based heart-rate estimation:** the paper proposes a high-resolution HR detection algorithm (ESPRIT via Hilbert Transform) that significantly outperforms standard FFT, autocorrelation, zero-crossing, and peak detection methods. This algorithm is not yet implemented in the firmware — it is the primary signal-processing goal for future development.
- **PPG signal model:** the paper models the PPG signal as $x[n] = A \cdot \cos(2\pi f n + \theta) + r[n]$, where f is the heart-rate frequency. HR estimation is a frequency-estimation problem solved by ESPRIT on the MAX30102 IR LED signal.
- **Optimal sensor configuration:** experiments showed a 100 Hz sampling rate with 1600 μ s LED pulse width (16-bit ADC resolution) gives the best HR estimation accuracy. The firmware's sensor init function sets 6.2 mA LED current — the paper recommends 27.1 mA for ESPRIT, so this constant should be raised when the algorithm is activated.
- **System power:** the paper reports 2.032 mA average current consumption, giving approximately 73 hours of battery life on a 150 mAh LiPo battery (see §9 for the calculation).

13. Conclusion

The Smartband V001 firmware (smartband_v001_firmware_masrur) is a well-structured, professionally organised embedded BLE firmware for the CC2640R2F microcontroller. The repository contains 70 files across 8 directories and is built on TI's SimpleLink SDK with the SYS/BIOS RTOS.

The firmware architecture is sound and complete in its infrastructure. The BLE peripheral role, GATT service with five characteristics, periodic 350 ms update cycle, button GPIO interrupt handling, SSD1306 OLED driver, and I2C sensor driver functions are all implemented and in place. The RTOS threading model using SYS/BIOS Tasks, Clocks, Events, and Queues is correctly structured.

The current limitation is that live sensor reading is commented out, and the device transmits placeholder data {0, 1, 2, 0, 0x77, 0x88} via BLE CHAR1. Activating real sensor measurements requires only a few lines of code to be uncommented in `SmartBand_periodicFxn()` and `SimplePeripheral_init()`, as detailed in §12.1.

The next steps are to install TI Code Composer Studio with the SimpleLink CC2640R2 SDK, build and flash the firmware onto the physical Band01_Rev1 hardware, activate the sensor calls, harden BLE security beyond Just-Works pairing, and begin functional testing of the complete sensor-to-BLE data pipeline. This will prepare the ground for implementing the ESPRIT-based heart-rate estimation algorithm described in the companion research paper.

References

- Pinheiro, G. P. M., Miranda, R. K., Praciano, B. J. G., Santos, G. A., Mendonça, F. L. L., Javidi, E., da Costa, J. P. J., & de Sousa, R. T. Jr. (2021). Multi-Sensor Wearable Health Device Framework for Real-Time Monitoring of Elderly Patients Using a Mobile Application and High-Resolution Parameter Estimation. *Frontiers in Human Neuroscience*, 15, 750591. <https://doi.org/10.3389/fnhum.2021.750591>
- Texas Instruments. (2019). CC2640R2F SimpleLink™ Bluetooth® Low Energy Wireless MCU. Datasheet (SWRS204). Texas Instruments.
- Texas Instruments. (2018). CC13x0, CC26x0 SimpleLink™ Wireless MCU Technical Reference Manual (SWCU117). Texas Instruments.
- Texas Instruments. (2020). SimpleLink™ CC2640R2 SDK BLE5-Stack User's Guide. Texas Instruments.
- Texas Instruments. (2017). TI-RTOS Kernel (SYS/BIOS) User's Guide (SPRUEX3). Texas Instruments.
- Maxim Integrated. (2018). MAX30102 High-Sensitivity Pulse Oximeter and Heart-Rate Sensor for Wearable Health. Datasheet Rev. 1. Maxim Integrated.
- Maxim Integrated. (2016). MAX30205 Human Body Temperature Sensor. Datasheet. Maxim Integrated.
- STMicroelectronics. (2010). LIS331DL MEMS Digital Output Motion Sensor. Datasheet. STMicroelectronics.
- Solomon Systech. (2008). SSD1306 Advance Information — 128×64 Dot Matrix OLED/PLED Segment/Common Driver with Controller. Datasheet Rev. 1.1. Solomon Systech.
- Aisler. (2026). Bill of Materials for Band01_Rev1 (PDF, March 2026). In repository: BOM/.
- Prochchhod, M. J. (2026). Health Wearable Device — PCB Component Analysis Report, Band01_Rev1 (Task 1). Hochschule Hamm-Lippstadt.